Faculty of Information Communication Technology

ITCS222 –Computer Organization and Architecture

Term Project: Assembly Programming

Presented by:

6388003 Phuriwat Angkoondittaphong

Submitted to:

Assoc. Prof. Dr. Chomtip Pornpanomchai

Dr. Ittion Rassameeroj

Semester 1 Academic year 2021

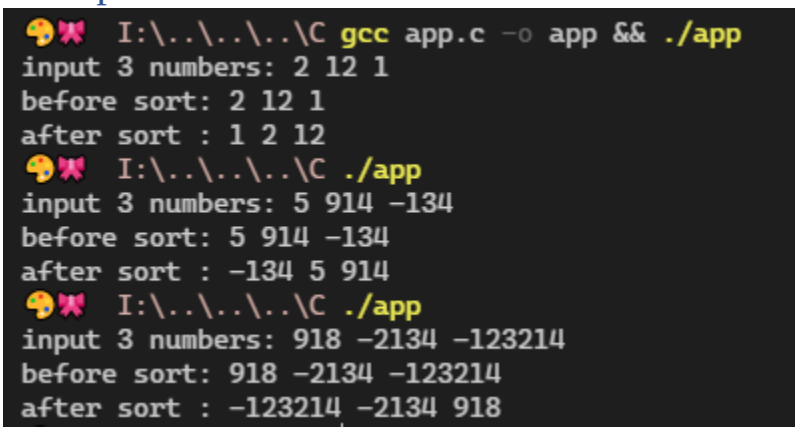
Contents

C code

```c
// gcc app.c -o app && ./app
#include <stdio.h>
#define len 3

int find_min_index(int arr[], int start, int end){
  int i;
  int current_min = start;
  for(i = start+1; i < end; i++){
    if(arr[i] < arr[current_min]){
      current_min = i;
    }
  }
  return current_min;
}

void sort_array(int arr[], int start, int end){
  int i, j, t;
  for(i = start; i < end; i++){
    j = find_min_index(arr, i, end);
    t = arr[j];
    arr[j] = arr[i];
    arr[i] = t;
  }
}


void print_array(int arr[], int start, int end){
  for(int i = start; i < end; i++){
    printf("%d ", arr[i]);
  }
  printf("\n");
}

int main(){

  int a[len];
  int i;
  printf("input 3 numbers: ");
  for(i = 0; i < len; i++){
    scanf("%d", &a[i]);
  }
  printf("before sort: ");
```

```
  print_array(a, 0, len);
  sort_array(a, 0, len);
  printf("after sort : ");
  print_array(a, 0, len);
  return 0;
}
```

C output



If you want to run this, try executing app.exe in C folder (windows only).

Built it on your own OS using `gcc app.c -o app`

NASM code

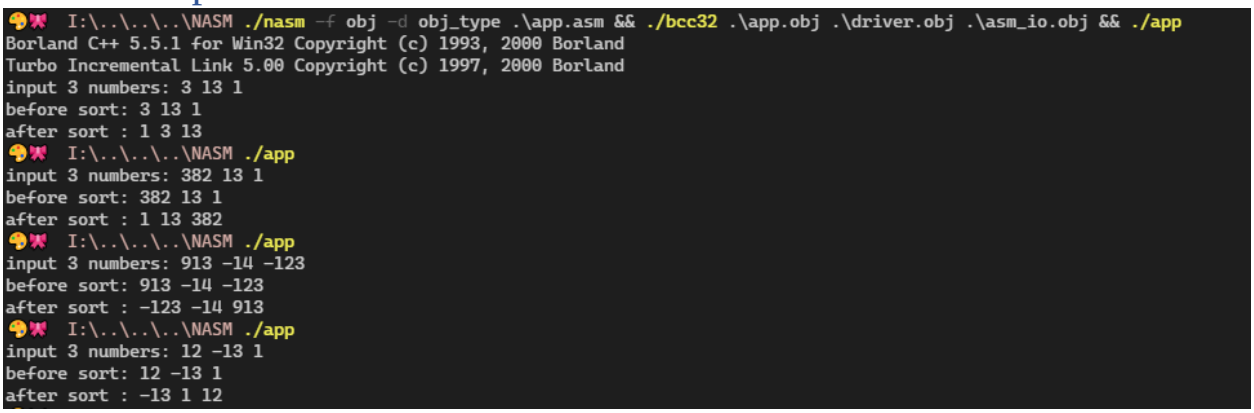
```
; ./nasm -f obj -
d obj_type .\app.asm && ./bcc32 .\app.obj .\driver.obj .\asm_io.obj && ./a
pp

%include "asm_io.inc"

segment _DATA public align=4 class=DATA use32
  INPUT_TEXT db 'input 3 numbers: ', 0
  BEFORE_TEXT db 'before sort: ', 0
  AFTER_TEXT db 'after sort : ', 0
  SPACE   db ' ', 0

  INT_SIZE equ 4          ; size of integer's author using below


segment _BSS public align=4 class=BSS use32
  endarr resd  1          ; end of the array that author want it to be (defaul
t is 3)
  i   resd  1          ; typical loop variable
  j   resd  1          ; typical loop variable
  min_i resd  1          ; max index, will use in find max
  num  resd  3          ; array
  t   resd  1          ; tempory for swapping operation

group DGROUP _BSS _DATA


segment _TEXT public align=1 class=CODE use32
   global _asm_main
_asm_main:
  enter  0,0            ; setup routine
  pusha


  mov dword [endarr], 3
  mov dword [i], 0

  mov eax, INPUT_TEXT
  call print_string
while_input:

  ; read from stdin
```

```
  mov ecx, [i]
  call read_int
  mov [num + ecx * INT_SIZE], eax    ; sizeof(dtype) = 4

  mov ecx, [i]
  inc ecx
  mov [i], ecx
  mov ebx, [endarr]
  cmp ebx, ecx
  jg while_input            ;end of while_input


  mov eax, BEFORE_TEXT
  call print_string
; init for outputing array
  ; set i = 0
  mov dword [i], 0
while_output1:
  ; calculate for num[i] = num + sizeof(dtype) * i
  mov ecx, [i]
  mov eax, [num + ecx * INT_SIZE]    ; sizeof(dtype) = 4
  call print_int
  mov eax, SPACE
  call print_string

  ; loop condition
  inc ecx
  mov [i], ecx
  mov ebx, [endarr]
  cmp ecx, ebx              ; if i < endarr: goes up
  jl while_output1
  ; end of while_output1

  call print_nl

; find max and replace here
  mov eax, 0
  mov [i], eax
selection_sort:
  mov eax, [i]
  mov ebx, [endarr]
  cmp eax, ebx
  jge end_sort
```

```
; init for loop_find_min
  mov ecx, [i]
  mov [min_i], ecx
  inc ecx
  mov [j], ecx
loop_find_min:
  mov eax, [endarr]
  mov ecx, [j]
  cmp ecx, eax
  jge end_loop_find_min        ; if j < endarr: do below, else: jmp to the en
d

  mov ecx, [min_i]
  mov ebx, [num+ ecx * INT_SIZE]    ; get current small value
  mov ecx, [j]
  mov eax, [num+ ecx * INT_SIZE]    ; get `this` value
  cmp eax, ebx
  jge if_not_new_min          ; if num[j] < num[min_i]: do below, else: skip
below

; if_new_max
  mov [min_i], ecx          ; min_i <- i ; set a new small

if_not_new_min:              ; and after set new max
  ; mov ecx, [j]
  inc ecx               ; assumn ecx = j
  mov [j], ecx             ; j++
  jmp loop_find_min
                  ; ends loop_find_min here

end_loop_find_min:

  ; swap num[i] <=> num[min_i]
  mov ecx, [i]
  mov eax, [num+ ecx * INT_SIZE]
  mov ecx, [min_i]
  mov ebx, [num+ ecx * INT_SIZE]

  mov [num+ ecx * INT_SIZE], eax    ; at write at num[min_i] <- num[i]
  mov ecx, [i]
  mov [num+ ecx * INT_SIZE], ebx    ; at write at num[i] <- (old) num[min_i]
```

```
  ; i++
  inc ecx
  mov [i], ecx
  jmp selection_sort

end_sort:


  mov eax, AFTER_TEXT
  call print_string
; init for outputing array
  ; set i = 0
  mov dword [i], 0
while_output2:
  mov ecx, [i]
  mov eax, [num+ ecx * INT_SIZE]    ; calculate for num[i] = num + sizeof(dty
pe) * i
  call print_int
  mov eax, SPACE
  call print_string

  ; loop condition
  inc ecx
  mov [i], ecx
  mov ebx, [endarr]
  cmp ecx, ebx              ; i < endarr
  jl while_output2          ; end of while_output2

  popa
  mov eax, 0
  leave
  ret                 ; return 0
```

NASM output



If you want to run this, try executing app.exe in NASM folder.

Built it on your own using `./nasm -f obj -d obj_type ./app.asm && ./bcc32 ./app.obj ./driver.obj ./asm_io.obj `

(windows only)